



UNIVERSITY OF
LEICESTER

**School of Computing and
Mathematical Sciences**

**CO7201 Individual Project
Preliminary Report**

**An AI- Assisted Recipe Generation and
Meal Personalisation System**

Sumit Nandu Yadav

sny2@student.le.ac.uk

Student Id - 249021470

Project Supervisor: Dr. Vladimirova Tanya

Second Marker: Dr. Lau Newman

Word Count: 2033

26/06/2026

DECLARATION

All sentences or passages quoted in this report, or computer code of any form whatsoever used and/or submitted at any stages, which are taken from other people's work have been specifically acknowledged by clear citation of the source, specifying author, work, date and page(s). Any part of my own written work, or software coding, which is substantially based upon other people's work, is duly accompanied by clear citation of the source, specifying author, work, date and page(s). I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this module and the degree examination as a whole.

Name: Sumit Yadav

Date:26/06/2026

Table of Contents

1. Aims and Objectives.....	3
1.1 Project Motivation and Relevance	3
1.2 Aim of the Project.....	3
1.3 Objectives and Main Challenges	3
2. Requirements	3
2.1 Essential Requirements	3
2.2 Recommended Requirements	4
2.3 Optional Requirements	4
3. Technical Specification	4
3.1 System Architecture and Technologies.....	4
3.2 APIs, Database and Security Considerations.....	4
4. Requirements Evaluation Plan	5
4.1 Evaluation Criteria	5
4.2 Testing Methods and User Involvement	5
5. Background Research and Reading List	5
5.1 Background Research.....	5
5.2 Reading List	6
6. Time-plan and Risk Plan	6
6.1 Time-plan and Milestones.....	6
6.2 Risk Plan.....	6
7. References.....	7

1. Aims and Objectives

1.1 Project Motivation and Relevance

CulinaAI is proposed as an AI- assisted recipe generation and meal personalisation system designed to help users create practical meal ideas from available ingredients and personal preferences. Many users face difficulty deciding what to cook when they have limited ingredients, dietary restrictions, allergy concerns, time constraints or different cooking skills levels. Traditional recipe websites usually provide large collection of fixed recipes, but they often require users to search manually and may not adapt well to individual requirements. This project is worth developing because it explores how generative AI can be integrated into a web application to provide structured and personalised recipe suggestions that are more relevant to the user's situation.

1.2 Aim of the Project

The main aim of this project is to design, implement and evaluate a web-based system that can generate recipe suggestions, allergy information, cooking time and difficulty level. The proposed system will focus on more than simple recipe display. It will aim to provide a complete recipe workflow where users can generate recipe idea, manage saved outputs, revisits previous meals and interact with the system through account-based features,

1.3 Objectives and Main Challenges

The key objectives are to identify clear functional and non-functional requirements, design a suitable web application architecture, integrate an AI model for structured recipe generation, support recipe personalisation, and evaluate the system through testing and user feedback. The project will also consider features such as saved recipes, favourites, feedback and recipe refinement where feasible within the project timetable. The main challenges include producing consistent AI outputs, handling allergy-related information responsibly, controlling external API usage, protecting user data, and ensuring that the system remains realistic within the available development time.

2. Requirements

2.1 Essential Requirements

The essential requirements define the minimum functionality required for CulinaAI to meet its main aim. The system must provide user registration, login and logout so that recipe activity can be associated with individual users. It must allow users to enter available ingredients and choose relevant constraints such as cuisine type, meal type, dietary preference, allergy information, cooking time and difficulty level. Based on these inputs, the system must generate a structured recipe suggestion containing a recipe title, ingredient list, cooking steps, estimated time and serving information. The application must also store generated recipes in a database so that users can access saved recipes later. Basic validation, error handling, responsive interface design and secure handling of API keys must be included.

2.2 Recommended Requirements

Recommended requirements extend the usefulness of the system and improve the user experience. The system should allow users to mark recipes as favourites and view them separately from the general saved recipe list. It should support recipe feedback or rating so that user responses can be considered during evaluation. The system should also allow users to modify or regenerate a recipe by providing additional instructions, such as making it healthier, easier, spicier or suitable for a different dietary preferences. Search and filtering options should be included for saved recipes where possible, helping users find recipe outputs more easily.

2.3 Optional Requirements

Optional requirements will be considered if the core recommended requirements are completed within the project timetable. These may include email sharing of recipes, print or export functionality, a generated shopping list based on recipe ingredients, and a nutrition-related feature using an external nutrition API or simplified nutritional estimation. These features would add value to the final system, but they are not essential for proving the main concept. If time or API limitations occur, the optional features can be reduced or replaced by a simpler alternative without affecting the core aim of the project.

3. Technical Specification

3.1 System Architecture and Technologies

CulinaAI will be developed as a web-based application using the Django framework. Django is suitable for this project because it supports rapid development, user authentication, database integration, form handling and a model-view-template architecture. The front end will use Django templates with HTML, CSS, JS and Bootstrap to create a responsive user interface. The system will follow a modular design where user accounts, recipe generation, saved recipes and supporting features are organised into separate application components. GitLab will be used for version control, allowing progress to be tracked through regular commits.

3.2 APIs, Database and Security Considerations

The project will use PostgreSQL as the main relational database to store user accounts, recipe inputs, generated outputs, saved recipes, favourites and feedback data. AI-ASSISTED RECIPE GENERATION WILL BE IMPLEMENTED THROUGH AN EXTERNAL LARGE LANGUAGE MODEL API, with carefully designed prompts used to convert user preferences into structured recipe suggestions. Environment variables will be used to store API keys and email credentials securely, rather than hard coding sensitive information into the source code. The system will also use Django's built-in security features, including password hashing, CSRF protection and authenticated access control, to reduce common web application risks. If time permits, an external nutrition API may be investigated as an optional extension.

4. Requirements Evaluation Plan

4.1 Evaluation Criteria

The system will be evaluated against the requirements identified for the project. The main criteria will include whether users can successfully register and login, enter recipe preferences, generate a structure recipe output, save and revisit recipes, and manage recipes through features such as favourites and feedback where implemented. The quality of AI-generated recipes will be assessed by checking whether the output is relevant to the user's ingredients, dietary preferences, allergies, cuisine choice, cooking time and difficulty level. Usability will also be considered, including whether the interface is clear, responsive and easy to navigate.

4.2 Testing Methods and User Involvement

Functional testing will be used to check that each major feature works as expected, including form validation, authentication, recipe generation, database storage, saved recipe access and error handling. Manual browser testing will be carried out on different sizes to check responsiveness and layout consistency. User involvement will include a small group of test users, such as classmates or other students, who will be asked to complete typical tasks including generate a recipe, saving it, viewing it later and giving feedback on the usefulness of the result. Supervisor feedback will also be used to refine the system during development. The project will be considered successful if the essential requirements are implemented, the system can generate relevant personalised recipes, user data is stored correctly, and evaluation feedback shows that the application is usable and helpful.

5. Background Research and Reading List

5.1 Background Research

The background research for CulinaAI focuses on three main areas: AI-assisted content generation, personalised recommendations systems, and web application development. Generative AI is relevant because the project depends on converting user inputs, such as ingredients and dietary preferences, into structured recipe suggestions. Research and guidance on prompt engineering is important because the quality of the recipe output will depend on how clearly the system instructs the AI model and how consistently the response is structured (OpenAI 2026).

Personalisation is also central to the project. Recommender system literature shows that useful digital systems should consider user preferences, context and intent rather than only presenting generic results (Ricci, Rokach and Shapira, 2015). In CulinaAI, this idea is applied through ingredients, cuisine type, meal type, dietary preferences, allergies, cooking time and difficulty level. However, this also creates challenges, including how to make outputs relevant, how to avoid unsafe allergy-related suggestions, and how to evaluate whether the generated recipes are genuinely useful to users.

The technical background includes Django web development, relational database design, authentication, form validation and secure handling of sensitive configuration data. Django is relevant because it provides support for web development tasks such as authentication, forms and secure request handling (Django Software Foundation, 2026). PostgreSQL is relevant for storing users accounts, recipe records, favourites and feedback because it is a relational database system suitable

for structured application data (PostgreSQL Global Development Group, 2026). Usability and software evaluation material will also be reviewed to support the design of responsive and understandable user interface (Nielsen, 1993).

5.2 Reading List

The initial reading list will include official Django documentation for authentication, forms, templates and security, PostgreSQL documentation for relational database design and querying, OpenAI API documentation and prompt engineering guidance for structured AI output generation, research literature on recommender systems and personalisation, and project management guidance such as Dawson’s work on computing projects. These sources will support both the design decision and the evaluation approach used in the project.

6. Time-plan and Risk Plan

6.1 Time-plan and Milestones

The project will be developed incrementally between late June and early September. The first priority will be to confirm the requirements, finalise the design and produce a working core prototype before extending the system with recommended and optional features. This phased approach will help ensure that the essential functionality is completed first.

Period	Main Work	Milestone
Late June	Finalise preliminary report, requirements and design	Preliminary report submitted
Early July	Set up Django structure, authentication, database models and recipe input forms	Core application structure working
Mid July	Integrate AI recipe generation and store generated recipe outputs	Functional recipe generation prototype
Late July	Add saved recipes, favourites, feedback and prepare for interim review	Usable prototype ready for feedback
August	Add recommended features such as recipe modification, search/filtering, sharing or export where feasible	Enhanced prototype and evaluation evidence
Late August – Early September	Complete testing, evaluation, final report, code clean-up and viva preparation	Final submission completed

6.2 Risk Plan

The main risk is that AI-generated recipes may be inconsistent, incomplete or not closely aligned with the user’s preferences. This will be reduced by improving prompt design, testing different user input combinations and checking whether outputs follow the expected recipe structure. Another risk is

external API cost, failure or access limitation. To manage this, API usage will be controlled during development and optional API-based features can be simplified if required.

A further risk is project scope becoming too large. This will be managed by prioritising essential requirements first, then recommended features, and only attempting optional features if time allows. Technical integration issues may also occur as the database, authentication and recipe features grow. Regular GitLab commits, incremental testing and supervisor feedback will be used to identify problems early. If user evaluation is limited by time, a small structured usability test with students and supervisor feedback will be used as an alternative.

7. References

- 1) Dawson, C. W. (2009) Projects in Computing and Information Systems: A Student's Guide. 2nd edn. Harlow: Addison-Wesley.
<https://www.semanticscholar.org/paper/Projects-in-Computing-and-Information-Systems%3A-A-Dawson/6c305884e237e2dd4ca9cb77ef6c1d7694e9bf27>
- 2) Django Software Foundation (2026) Django Documentation.
<https://docs.djangoproject.com/en/6.0/>
- 3) PostgreSQL Global Development Group (2026) PostgreSQL Documentation.
<https://www.postgresql.org/docs/>
- 4) OpenAI (2026) OpenAI API Documentation and Prompt Engineering Guidance.
<https://developers.openai.com/api/docs/guides/prompt-engineering>
- 5) Nielsen, J. (1994) Usability Engineering. Morgan Kaufmann.
<https://www.nngroup.com/books/usability-engineering/>